

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA15
---------------------	---	---------------------	-------

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 50

Code of Conduct

I C. Pavani/(192524260)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C.Pavani

Assessment Tasks

Analytical Problem Solving

1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.
2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.
3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.
4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.
5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Analytical Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 – Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Interpretation	20%	Interprets all technical requirements accurately	Interprets most requirements correctly	Partial interpretation	Misinterprets the problem
Analytical Reasoning	25%	Strong reasoning with clear cause-effect analysis	Good reasoning with minor gaps	Basic analysis only	Weak or no analysis
Method Selection	20%	Chooses highly appropriate virtualization and security concepts	Chooses mostly suitable concepts	Partially suitable concepts	Inappropriate approach
Solution Quality	20%	Provides feasible and technically sound solution	Reasonable solution with minor issues	Basic solution with limitations	Unclear or invalid solution
Presentation of Analysis	15%	Well-structured and technically precise	Generally clear	Somewhat unclear	Difficult to follow

Total Marks Awarded:

Question 1:

Q1. Compare Type-1 and Type-2 Hypervisors for a Cloud Datacenter Hosting Mission-Critical Applications

Problem

Interpretation:

A cloud data center needs a hypervisor that provides high performance, strong security, and easy management for critical applications.

Analysis:

Feature	Type-1 Hypervisor	Type-2 Hypervisor
Performance	High (runs directly on hardware)	Lower (runs on host OS)
Security	Strong, smaller attack surface	Weaker due to host OS dependency
Manageability	Easy centralized management	Better for personal/testing use
Reliability	Very high	Moderate

Analytical Reasoning:

- Type-1 hypervisors access hardware directly, reducing overhead and improving performance.
- They provide better isolation between virtual machines, improving security.
- They support enterprise features such as live migration, high availability, and centralized management.
- Type-2 hypervisors depend on the host operating system, making them slower and more vulnerable.

Solution:

For a mission-critical cloud data center, Type-1 Hypervisor is the best choice because it provides:

- Better performance

- Strong security
- High reliability
- Easy management
- Efficient resource utilization

Conclusion:

Type-1 Hypervisor is the recommended solution because it meets the performance, security, and manageability requirements of mission-critical cloud applications.

Code:

```
print("Type-1 Hypervisor")
print("Performance : High")
print("Security : High")
print("Manageability: Easy")

print("\nType-2 Hypervisor")
print("Performance : Medium")
print("Security : Medium")
print("Manageability: Moderate")

print("\nBest Choice: Type-1 Hypervisor")
print("Reason: Better performance, security, and reliability for mission-critical cloud applications.")
```

output screen shot:

The screenshot shows the Programiz Python Online Compiler interface. The code in the editor is as follows:

```
1 print("Type-1 Hypervisor")
2 print("Performance : High")
3 print("Security : High")
4 print("Manageability: Easy")
5
6 print("\nType-2 Hypervisor")
7 print("Performance : Medium")
8 print("Security : Medium")
9 print("Manageability: Moderate")
10
11 print("\nBest Choice: Type-1 Hypervisor")
12 print("Reason: Better performance, security, and reliability for mission-critical cloud applications.")
```

The output of the code execution is displayed on the right side:

```
Type-1 Hypervisor
Performance : High
Security : High
Manageability: Easy

Type-2 Hypervisor
Performance : Medium
Security : Medium
Manageability: Moderate

Best Choice: Type-1 Hypervisor
Reason: Better performance, security, and reliability for mission-critical cloud applications.

=== Code Execution Successful ===
```

Question 2:

Workload Isolation Using Virtualization

Problem Statement:

A cloud service provider hosts three applications:

- Finance Application
- Healthcare Records
- Student Portal

Analyze how virtualization isolates these workloads and identify two major attack surfaces.

Analytical Solution

Step 1: Identify the Requirement

The cloud provider hosts different applications that contain sensitive and confidential data. These applications should not interfere with each other even though they run on the same physical server.

Step 2: Analyze the Role of Virtualization

Virtualization creates separate Virtual Machines (VMs) for each application.

- Finance application runs in VM-1.
- Healthcare application runs in VM-2.
- Student portal runs in VM-3.

Each VM has its own operating system, memory, CPU allocation, storage, and applications. Because of this isolation, a failure or attack in one VM does not directly affect the others.

Step 3: Workload Isolation

Virtualization provides isolation through:

- Separate Virtual Machines for each workload.
- Independent operating systems.
- Dedicated virtual memory and CPU allocation.
- Virtual network segmentation.
- Access control for each VM.

This ensures better security, reliability, and efficient resource utilization.

Step 4: Identify Two Major Attack Surfaces

1. Hypervisor Attack

- If an attacker compromises the hypervisor, all virtual machines running on it may be affected.
- This is one of the most critical security risks.

2. VM Escape Attack

- A malicious program inside one virtual machine escapes its isolation and accesses another VM or the host system.
- This can lead to unauthorized access and data leakage.

Conclusion:

Virtualization isolates finance, healthcare, and student workloads by running them in separate virtual machines with independent resources. The two major attack surfaces are Hypervisor Attacks and VM Escape Attacks. Therefore, proper hypervisor security, access control, and regular updates are essential to maintain a secure cloud environment.

Code:

Q2: Workload Isolation in Cloud Virtualization

```
workloads = ["Finance", "Healthcare", "Student Portal"]
```

```
print("Cloud Workload Isolation")
print("-----")
```

```
for vm in workloads:
    print(vm, "-> Isolated in a separate Virtual Machine")
```

```
print("\nMajor Attack Surfaces:")
print("1. Hypervisor Attack")
print("2. VM Escape Attack")
```


workloads, and using faster storage will improve overall performance and reduce VM boot time.

Code:

Q3: VM Host Utilization Analysis

```
cpu = 92
memory = 88
latency = 35
boot_time = 170

print("VM Host Utilization Analysis\n")

if cpu > 90:
    print("CPU Bottleneck: High CPU utilization")

if memory > 80:
    print("Memory Bottleneck: High memory usage")

if latency > 30:
    print("Storage Bottleneck: High I/O latency")

if boot_time > 120:
    print("VM Boot Time: Too slow")

print("\nCorrective Actions:")
print("- Increase CPU resources")
print("- Add more RAM")
print("- Use SSD/NVMe storage")
print("- Balance workloads using VM migration")
output:
```

The screenshot shows a web browser window with the URL `programiz.com/python-programming/online-compiler/`. The page features a dark-themed interface for the Programiz Python Online Compiler. At the top, there are banners for Adobe Creative Cloud Pro and a "Programiz PRO" subscription. The main area is divided into a code editor on the left and an output window on the right. The code editor contains a Python script for VM Host Utilization Analysis, which checks for bottlenecks in CPU, memory, storage, and boot time, and provides corrective actions. The output window shows the results of the script execution, indicating that all checks passed successfully. The bottom of the browser window shows a Windows taskbar with various application icons and system status indicators.

```
main.py
8 print("VM Host Utilization Analysis\n")
9
10 if cpu > 90:
11     print("CPU Bottleneck: High CPU utilization")
12
13 if memory > 80:
14     print("Memory Bottleneck: High memory usage")
15
16 if latency > 30:
17     print("Storage Bottleneck: High I/O latency")
18
19 if boot_time > 120:
20     print("VM Boot Time: Too slow")
21
22 print("\nCorrective Actions:")
23 print("- Increase CPU resources")
24 print("- Add more RAM")
25 print("- Use SSD/NVMe storage")
26 print("- Balance workloads using VM migration")
```

Output

```
VM Host Utilization Analysis

CPU Bottleneck: High CPU utilization
Memory Bottleneck: High memory usage
Storage Bottleneck: High I/O latency
VM Boot Time: Too slow

Corrective Actions:
- Increase CPU resources
- Add more RAM
- Use SSD/NVMe storage
- Balance workloads using VM migration

=== Code Execution Successful ===
```

Question 4:

Question 4: Analytical Problem Solving

Question: Analyze how Software Defined Datacenter (SDDC) architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.

Problem Interpretation

The task is to compare a traditional datacenter with a Software Defined Datacenter (SDDC) and analyze how SDDC improves flexibility and resource management.

Analysis

Feature Traditional Datacenter Software Defined Datacenter (SDDC)

Compute	Physical servers	Virtual machines
Storage	Fixed physical storage	Virtualized storage
Network	Hardware-based configuration	Software-defined networking
Resource Allocation	Manual	Automatic
Scalability	Slow	Fast
Management	Complex	Centralized

Analytical Reasoning

Compute Virtualization: Multiple virtual machines run on one physical server, improving resource utilization.

Storage Virtualization: Storage is pooled and allocated dynamically, making management easier.

Network Virtualization: Networks are created and managed through software, enabling faster deployment and configuration.

SDDC automates resource provisioning, reducing manual effort and improving agility.

Solution

A Software Defined Datacenter provides:

1. Faster deployment of resources.
2. Better utilization of hardware.
3. Centralized management.
4. Easy scalability.
5. Reduced operational cost.

Conclusion

Compared to a traditional datacenter, Software Defined Datacenter (SDDC) offers greater agility through compute, storage, and network virtualization, making cloud infrastructure more flexible, scalable, and efficient.

Code:

Q4: Software Defined Datacenter (SDDC) Analysis

```
print("Software Defined Datacenter (SDDC)")
print("-----")

print("Compute Virtualization : Faster VM deployment")
print("Storage Virtualization : Flexible storage allocation")
print("Network Virtualization : Software-defined networking")
```



```

print("\nAdvantages of SDDC:")
print("1. Fast resource provisioning")
print("2. Centralized management")
print("3. Better scalability")
print("4. Efficient resource utilization")
print("5. Lower operational cost")

print("\nConclusion:")
print("SDDC provides better agility than a traditional datacenter.")

```

output:

The screenshot shows a web browser window with the URL `programiz.com/python-programming/online-compiler/`. The page features a dark-themed interface with a sidebar on the left containing icons for various programming languages (Python, JavaScript, TypeScript, etc.). The main area is divided into two sections: a code editor on the left and an output console on the right.

The code editor contains the following Python code:

```

1 # Q4: Software Defined Datacenter (SDDC) Analysis
2
3 print("Software Defined Datacenter (SDDC)")
4 print("-----")
5
6 print("Compute Virtualization : Faster VM deployment")
7 print("Storage Virtualization : Flexible storage allocation")
8 print("Network Virtualization : Software-defined networking")
9
10 print("\nAdvantages of SDDC:")
11 print("1. Fast resource provisioning")
12 print("2. Centralized management")
13 print("3. Better scalability")
14 print("4. Efficient resource utilization")
15 print("5. Lower operational cost")
16
17 print("\nConclusion:")
18 print("SDDC provides better agility than a traditional
    datacenter.")

```

The output console displays the following results:

```

Software Defined Datacenter (SDDC)
-----
Compute Virtualization : Faster VM deployment
Storage Virtualization : Flexible storage allocation
Network Virtualization : Software-defined networking

Advantages of SDDC:
1. Fast resource provisioning
2. Centralized management
3. Better scalability
4. Efficient resource utilization
5. Lower operational cost

Conclusion:
SDDC provides better agility than a traditional datacenter.

=== Code Execution Successful ===

```

Question 5:

Question: A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts. 💎

Problem Interpretation

The project failed because different cloud providers use different identity management systems. The task is to identify the problem and propose a secure federation-based solution.

Analysis

Issue

Analysis

Identity Mismatch

Different cloud providers use different authentication methods, causing login failures.

Security Risk

Users may gain unauthorized access if identities are not managed properly.

Privacy Issue

User information may be exposed during data sharing between cloud providers.

Analytical Reasoning

Identity mismatches prevent users from accessing services across multiple clouds.

A common identity management system is required for secure authentication.
Federation allows trusted identity sharing between cloud providers while protecting user privacy.

Solution

Use Identity Federation between cloud providers.
Implement Single Sign-On (SSO) for one-time authentication.
Use secure protocols such as SAML or OAuth/OpenID Connect.
Apply Role-Based Access Control (RBAC) to restrict access.
Encrypt user data and communication to ensure privacy.

Conclusion

A secure identity federation with SSO, SAML/OAuth, RBAC, and encryption eliminates identity mismatches and improves security and privacy in a multi-cloud environment.

Code:

Question 5: Multi-Cloud Identity Federation

```
clouds = ["AWS", "Azure", "Google Cloud"]

print("Multi-Cloud Identity Federation")
print("-----")

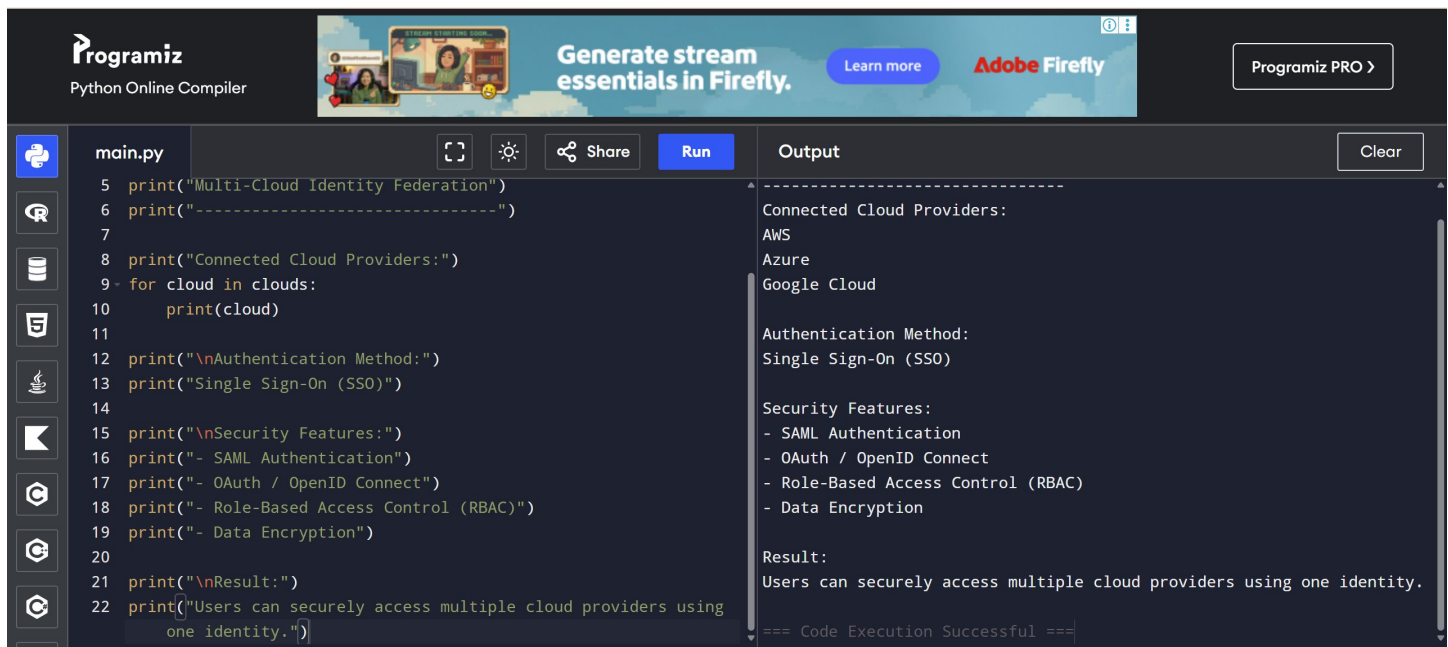
print("Connected Cloud Providers:")
for cloud in clouds:
    print(cloud)

print("\nAuthentication Method:")
print("Single Sign-On (SSO)")

print("\nSecurity Features:")
print("- SAML Authentication")
print("- OAuth / OpenID Connect")
print("- Role-Based Access Control (RBAC)")
print("- Data Encryption")

print("\nResult:")
print("Users can securely access multiple cloud providers using one identity.")
```

output:



The screenshot shows the Programiz Python Online Compiler interface. The top header includes the Programiz logo, a banner for 'Generate stream essentials in Firefly' with a 'Learn more' button and the Adobe Firefly logo, and a 'Programiz PRO >' button. The main area is divided into two panels. The left panel, titled 'main.py', contains the Python code from the previous block. The right panel, titled 'Output', shows the execution results. The output is formatted with dashed lines separating sections: 'Connected Cloud Providers:' lists AWS, Azure, and Google Cloud; 'Authentication Method:' shows Single Sign-On (SSO); 'Security Features:' lists SAML Authentication, OAuth / OpenID Connect, Role-Based Access Control (RBAC), and Data Encryption; and 'Result:' shows the message 'Users can securely access multiple cloud providers using one identity.'. At the bottom of the output panel, it says '=== Code Execution Successful ==='.